

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1001	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	N.Adi Narayana	Reg. No.:	192525346
Date:	01-10-2026	Duration:	60 minutes

Code of Conduct

I N.Adi Narayana/ 1925253463 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __ADI_____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

Customer Support Ticket Classification Using NLP and Machine Learning

1.Understand the Problem Statement

Problem Explanation

Organizations receive a constant stream of customer support tickets through email, chat, web forms and social media. Each ticket is free, unstructured text that an agent must read, classify (billing, technical, refund, account access), prioritize and route. Done manually, this is slow, inconsistent and does not scale, and critical issues such as outages or payment failures can get buried in the queue.

The problem is to build a system that uses Natural Language Processing (NLP) and Machine Learning (ML) to read each ticket and automatically assign a category and priority, so it reaches the right team with minimal human effort.

- **Input:** raw ticket text (subject and body), sometimes with metadata such as product or channel.

- **Output:** category, priority (Low, Medium, High, Urgent) and optionally sentiment.
- **Task type:** supervised text classification – multi-class, or multi-label if a ticket has several issues.

Identify the objectives of the proposed system.

- Automatically classify tickets into predefined categories and priority levels with high accuracy.
- Pre-process text (tokenization, stop-word removal, lemmatization) and convert it to features using TF-IDF, word embeddings or transformer embeddings.
- Train and compare Logistic Regression, SVM, Random Forest, LSTM and BERT.
- Evaluate with accuracy, precision, recall and F1-score, using macro-F1 for imbalanced classes.
- Deploy the best model for automatic routing, sending low-confidence predictions to a human agent.

State the scope of the problem.

- In scope: text tickets from email, web forms, chat and social media; category and priority prediction (sentiment optional); data cleaning, training, evaluation, REST API deployment and dashboard; human review of low-confidence tickets and periodic retraining.
- Out of scope: voice calls, images and attachments; automatic replies or chatbot resolution; replacing human agents or the existing helpdesk/CRM; languages other than English in the first release.

2. Identify Key Stakeholders and Challenges

Identify all the stakeholders involved in the system.

- Customers
- Support agents
- Support team leads / managers
- Specialist departments (Billing, Technical, Account, Refund)
- Data scientists / ML engineers
- Developers / DevOps / QA
- Product owner / business management
- Security and compliance officer

Explain the roles and responsibilities of each stakeholder.

- Customers: raise tickets, describe issues clearly and give feedback; they expect a fast, correct response.
- Support Agents: resolve tickets, review low-confidence predictions and correct wrong labels (corrections become training data).
- Team Leads / Managers: define categories and priority rules, monitor SLAs and workload, plan staffing from dashboards.
- Specialist Departments: receive routed tickets and resolve them within agreed time limits.

- Data Scientists / ML Engineers: collect and label data, build and compare models, and retrain when performance drops.
- Developers / DevOps / QA: build the API and integrations, set up CI/CD, test, deploy and monitor.
- Product Owner / Management: set goals and budget, prioritize the backlog, approve releases and measure value.
- Security and Compliance Officer: protect customer data, control access and follow privacy regulations.

Discuss the major challenges and issues faced by the stakeholders.

Every stakeholder is affected by slow, inconsistent triage, and the project must also overcome several technical and data challenges.

- Customers: long waiting times, transfers between teams and repeating the same issue.
- Agents: time lost sorting tickets, repetitive work and inconsistent categorization.
- Managers: SLA breaches, poor visibility of trends and difficulty scaling the team.
- Business: higher support cost, lower satisfaction and risk of losing customers.
- Noisy text: typos, slang, abbreviations, signatures and mixed languages.
- Class imbalance: some categories have far more tickets, which biases the model.
- Ambiguity: one ticket may describe several issues; short messages like “not working!!” lack context.
- Evolving language: new products add new vocabulary and cause model drift.
- Privacy and label quality: tickets may contain personal data and historical labels may be inconsistent.

3. Analyze the Current Approach

Describe the existing system or current process.

Ticket handling is mostly manual, at best supported by simple keyword rules in the helpdesk tool.

- The customer submits a ticket via email, chat, web form or social media.
- The ticket lands in a shared inbox or helpdesk queue.
- A first-level agent reads it and manually assigns a category and priority (or the customer picks one from a drop-down).

Identify its strengths and limitations.

Strengths

- Humans understand context, tone and nuance and handle unusual cases.
- Easy to start: no development cost or technical setup.
- Agents can spot emotionally charged or urgent messages.
- A new type of issue can be handled immediately without retraining.

Limitations

- Slow response: agents spend time sorting instead of resolving.
- Misrouting: tickets bounce between teams, increasing resolution time.
- Inconsistency: different agents categorize the same issue differently.
- Poor scalability as ticket volume grows.

Analyze the problems or gaps that need to be addressed.

The core gap is the absence of an intelligent, learning system that triages every ticket instantly and consistently.

- Manual triage: delay before work starts → instant automatic classification.
- No learning from history: mistakes repeat → ML model retrained regularly.
- Subjective labelling: inconsistent labels → one consistent model for all tickets.

4. Propose a Suitable Solution Using Software Engineering Principles

Design a software-based solution for the given problem.

The proposed solution is an end-to-end pipeline that takes raw ticket text, cleans it, converts it to numerical features, classifies it with a trained model and routes the ticket automatically. Low-confidence predictions go to a human agent, whose correction is fed back as new training data. The layered architecture is shown below.

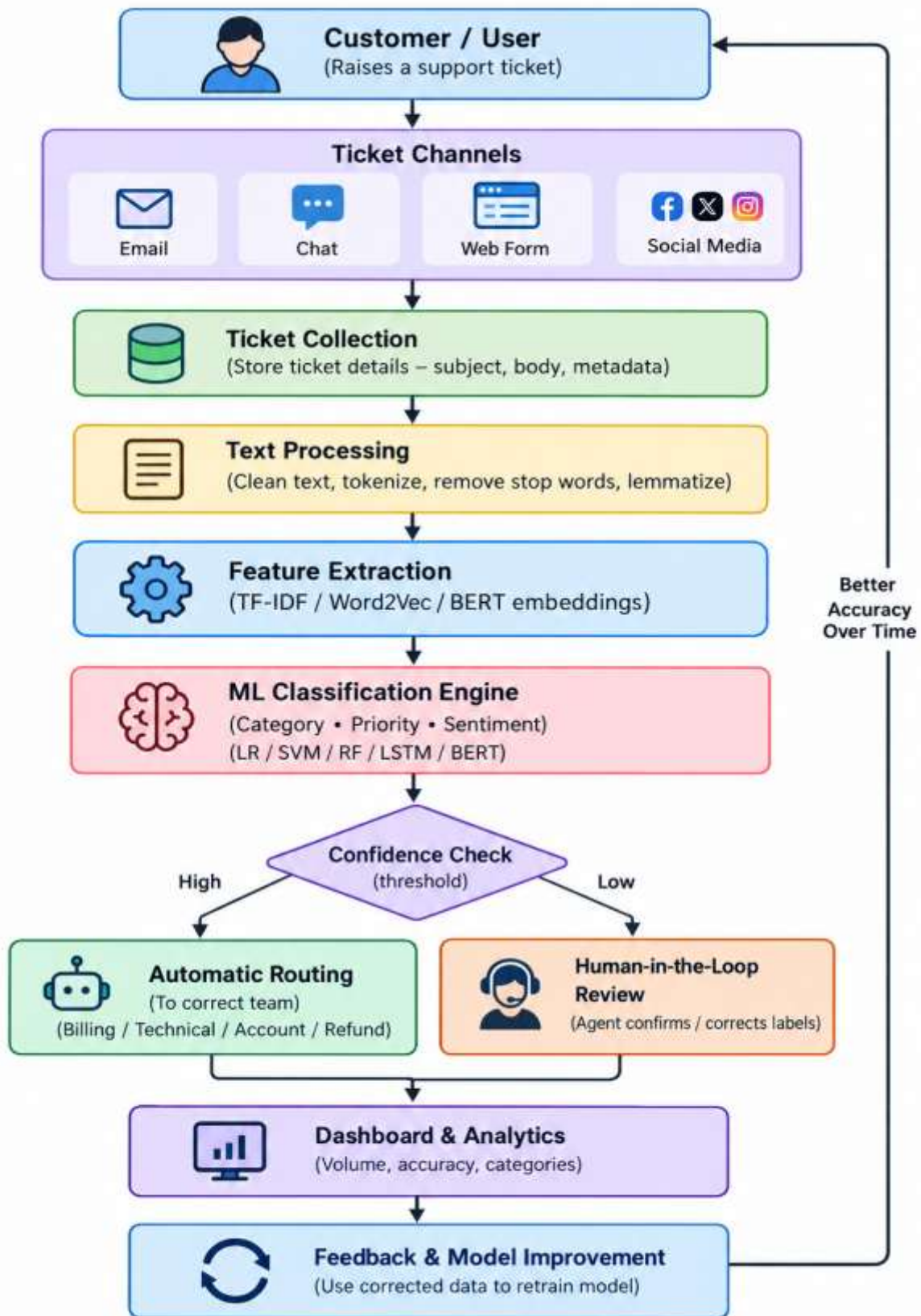


Figure 1: Architecture block diagram of the ticket classification and routing system

- Data: historical labelled tickets; public datasets such as Kaggle support tickets, CFPB Consumer Complaints or Banking77 for development.
- Models: TF-IDF + Logistic Regression/SVM baseline, then Random Forest/XGBoost, LSTM/CNN and fine-tuned DistilBERT/BERT; imbalance handled with class weights and SMOTE.
- Evaluation: stratified train/validation/test split (e.g. 70/15/15); accuracy, precision, recall, F1 and macro-F1; confusion matrix and error analysis.

Describe the major functional modules of the proposed system.

- Ticket Ingestion – collects tickets from all channels with subject, body and metadata.
- Text Preprocessing – lowercases, removes HTML, URLs and signatures, tokenizes, removes stop words and lemmatizes.
- Feature Extraction – TF-IDF with n-grams, Word2Vec/GloVe/FastText or BERT embeddings.
- Classification Engine – predicts category, priority and sentiment using the best model.
- Confidence and Routing – routes high-confidence tickets, flags low-confidence ones for review.
- Human-in-the-Loop Review – agents confirm or correct predictions; corrections saved as labelled data.
- REST API and Integration – FastAPI/Flask service for the helpdesk or CRM.
- Dashboard and Analytics – predictions, ticket volume by category and model accuracy.
- Monitoring and Retraining – tracks live accuracy, detects drift and retrains periodically.

Identify the functional and non-functional requirements.

Functional requirements

- Accept tickets from email, chat, web form and social media.
- Clean and pre-process ticket text automatically.
- Predict category and priority of every ticket with a confidence score.
- Route high-confidence tickets to the correct team and low-confidence ones to a human agent.
- Let agents correct predictions and store corrections as labelled data.

Non-functional requirements

- Performance – predict a ticket in about a second.
- Accuracy – high macro-F1 on all categories, with a target agreed with the product owner.
- Scalability – handle growing volume and peak loads without redesign.
- Availability / Reliability – 24x7 availability; fall back to the manual queue if the model service fails.

Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

- Modularity – ingestion, preprocessing, feature extraction, classification, routing and dashboard are separate modules with clear interfaces.

- Scalability – a stateless REST API in containers behind a load balancer; lighter models under heavy load, transformers where accuracy matters most.
- Maintainability – clean code, Git version control, versioned models, documentation and periodic retraining.
- Reliability – confidence threshold with human review, fallback to the manual queue, automated tests and monitoring.
- Usability – agents see category, priority and confidence on one screen and correct them in one click.
- Security – role-based access, encrypted storage and transport, personal-data masking and audit logs.

5. Justify the Chosen Methodology/Framework

Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

The selected framework is Agile – Scrum, supported by CI/CD and MLOps practices (automated testing, model versioning and monitoring). Work is done in two-week sprints led by a Product Owner, a Scrum Master and a cross-functional team, with sprint planning, daily stand-ups, review and retrospective.

- Sprint 0 – requirements, backlog, data collection and labelling guidelines, tool setup.
- Sprint 1 – data cleaning and preprocessing; class-distribution analysis.
- Sprint 2 – TF-IDF + Logistic Regression/SVM baseline; evaluation report; shadow mode.
- Sprint 3 – ensemble, LSTM and BERT models; imbalance handling; model comparison.
- Sprint 4 – REST API, confidence threshold, review screen and helpdesk integration.
- Sprint 5 – dashboard, security hardening, testing, release and monitoring.

Justify why the selected methodology/framework is suitable for the proposed system.

- Changing requirements: new products and issues appear; Scrum handles change through backlog refinement.
- Iterative ML work: the best model is found by experiments, from baseline to advanced.
- Early value: a baseline classifier can run in shadow mode in the first sprints.
- Continuous feedback: agents and managers review every increment.
- Risk control: short sprints expose data-quality and imbalance problems cheaply.
- Compared with others: Waterfall is not suitable (fixed requirements, costly changes) and Spiral is only partly suitable (heavy documentation for a small team).

Explain how it supports successful project development and maintenance.

- Each sprint ends with a tested, usable increment that meets the Definition of Done.
- Automated tests and CI/CD catch defects early and make releases repeatable.
- Version-controlled code and models allow quick rollback if a new model performs worse.

6. Discuss Expected Outcomes and Limitations

Describe the expected benefits of the proposed solution.

A TF-IDF baseline gives a solid starting point, while fine-tuned transformers improve accuracy on ambiguous and short tickets at the cost of more computing power.

- Faster first response because tickets are classified and routed instantly.
- Higher routing accuracy and consistency across tickets and agents.
- Less manual triage, so agents spend more time resolving issues.
- Better handling of urgent issues through priority prediction.
- Better scalability and clear analytics, leading to improved customer satisfaction.

Explain how the solution addresses the identified challenges.

- Slow response, manual triage: automatic classification and routing remove the sorting step.
- Misrouting and inconsistency: one model applies the same logic to all tickets.
- Missed urgency: priority prediction raises critical tickets first.
- Noisy text: cleaning, lemmatization and subword models (BERT).

Discuss potential risks, implementation challenges, and limitations.

The system supports agents but does not resolve tickets by itself, and its accuracy depends on the quantity and quality of labelled data.

- Poor or inconsistent labels: mitigated by labelling guidelines and error analysis.
- Model drift: continuous monitoring and periodic retraining.
- Wrong predictions on rare tickets: confidence threshold with human review.
- High cost of transformers: start with a baseline; use DistilBERT or quantization.

Suggest possible future enhancements.

- Support for multiple languages and mixed-language tickets.
- Automatic suggested replies, knowledge-base recommendations and chatbot integration.
- Active learning and explainable predictions (LIME/SHAP).
- SLA-breach prediction, voice and image/attachment tickets, and use of large language models.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5

Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25